Aperture — Low-Level Design

Module: Lists

Version: v1.0

Module Owner: Collection Platform

Dependencies:

- Authentication Module
- Users Module
- Metadata Module
- Notification Module (future)
- Discovery Module

---

1. Overview

The Lists module enables users to organize, curate, and share collections of movies and TV shows.

Rather than treating lists as simple arrays of content IDs, Aperture models them as first-class entities with their own metadata, ownership, permissions, and engagement.

Lists are intended to become one of the platform's strongest discovery mechanisms, alongside reviews and recommendations.

---

2. Design Goals

The Lists module should:

- Allow users to organize content
- Encourage curation and storytelling
- Support public and private collections
- Enable future collaborative editing
- Integrate with discovery and recommendations
- Scale independently from metadata

---

3. Module Ownership

Owns

- Lists
- Watchlists
- List items
- List ordering
- List visibility
- List metadata

Reads

- User profiles
- Movies
- TV shows

Never Modifies

- Metadata
- Reviews
- User profiles
- Recommendations

---

4. Responsibilities

The Lists module owns:

Personal Lists

Examples:

- Watchlist
- Favorites
- Completed
- Custom collections

Public Lists

Examples:

- Top 100 Horror
- Best Sci-Fi of the 2000s
- Comfort Movies

Ordering

Support:

- Manual ordering
- Automatic ordering (future)
- Ranking

Visibility

- Public
- Private
- Unlisted
- Collaborative (future)

Engagement

Future support:

- Likes
- Comments
- Shares
- Saves

---

5. High-Level Workflow

Create List

↓

Validate Request

↓

Create List

↓

Return List

Add Item

↓

Validate Movie

↓

Prevent Duplicate

↓

Insert Item

↓

Update Position

↓

Return Updated List

Remove Item

↓

Verify Ownership

↓

Remove Entry

↓

Recalculate Ordering

↓

Return Updated List

---

6. Data Flow

User

↓

API

↓

Validation

↓

List Service

↓

Repository

↓

Database

↓

Publish ListUpdated Event (future)

↓

Return Response

---

7. Data Model

Primary entities:

List

Stores:

- owner
- title
- description
- visibility
- timestamps
- cover image (future)

List Item

Stores:

- list
- movie / TV show reference
- position
- note (future)
- date added

Future entities:

- Collaborator
- List Like
- List Comment
- Saved List

---

8. Business Rules

Users may create multiple custom lists.

Watchlist is unique per user.

Duplicate items are not allowed within the same list.

List ownership determines modification rights.

Ordering must remain stable after insertions and deletions.

---

9. Validation Rules

List

- title required
- maximum title length
- description length

Items

- valid content ID
- duplicate prevention
- supported content type

Visibility

- valid visibility option
- collaborator permissions (future)

---

10. Database Interactions

Reads

- Retrieve list
- Retrieve user lists
- Retrieve watchlist
- Retrieve list items

Writes

- Create list
- Update list
- Delete list
- Add item
- Remove item
- Reorder items

Indexes should optimize:

- owner lookups
- visibility
- list item ordering

---

11. Discovery Integration

Public lists contribute to content discovery.

Examples:

- Trending lists
- Recently created lists
- Editorial picks
- Community-curated collections

The Discovery module consumes list metadata but does not own it.

---

12. Recommendation Integration

Lists provide strong preference signals.

Examples:

- Frequently grouped titles
- Favorite genres
- Preferred directors
- Franchise affinity

Recommendation systems consume these signals asynchronously.

---

13. Caching Strategy

Cache candidates:

- popular lists
- featured lists
- watchlists
- list summaries

Avoid caching lists that are edited frequently.

Invalidate cache after list modifications.

---

14. Performance Considerations

Potential hotspots:

- large public lists
- collaborative editing (future)
- trending collections

Mitigations:

- pagination
- lazy loading
- cached summaries
- indexed ordering

Reordering operations should avoid rewriting every list item whenever possible.

---

15. Security Considerations

Ownership checks are required for:

- edits
- deletion
- reordering
- visibility changes

Private lists should never appear in public discovery.

Future collaborative lists require role-based permissions.

---

16. Error Handling

Representative failures:

- list not found
- duplicate item
- unauthorized modification
- invalid content reference
- unsupported visibility

Errors should remain consistent with other modules.

---

17. Testing Strategy

Unit Tests

- duplicate prevention
- ordering logic
- visibility rules

Integration Tests

- list creation
- item insertion
- item removal
- reordering

API Tests

- CRUD operations
- watchlist endpoints
- pagination

Performance Tests

- large lists
- bulk operations
- concurrent edits (future)

---

18. Future Service Extraction

The Lists module is a moderate candidate for service extraction.

Reasons:

- independent domain
- predictable workload
- clean boundaries

Potential published events:

- ListCreated

- ListUpdated
- ListDeleted
- ListItemAdded
- ListItemRemoved

Potential consumers:

- Discovery
- Recommendation Engine
- Notifications
- Analytics

Service extraction is not expected until list traffic becomes a significant independent workload.

---

19. Future Evolution

Potential enhancements:

- Collaborative lists
- List templates
- AI-generated lists
- Smart dynamic lists
- Drag-and-drop organization
- Rich media in descriptions
- Follow lists
- Version history
- Export and import

The architecture should support these capabilities without breaking existing APIs.

---

20. Interview Discussion Points

Be prepared to explain:

- Why model Lists as independent entities?
- Why separate Watchlists from generic Lists?
- How would you efficiently reorder large lists?
- How would you implement collaborative editing?
- How do Lists contribute to recommendations?
- When would the Lists module justify becoming its own service?

---

21. Summary

The Lists module transforms collections into a meaningful part of Aperture's social and discovery ecosystem.

By treating lists as curated, shareable resources rather than simple collections, the module supports richer user expression, better discovery, and future collaborative experiences while maintaining clear ownership and modular boundaries.